

11-Annotation(注解)

鸣谢：黑马程序员



💡 Tip

Previous Chapter(上一章): 10-Reflection(反射)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

Next Chapter(下一章): 12-Dynamic Proxy(动态代理)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

• 11-Annotation(注解)

一、概述

二、自定义注解

1.格式

2.特殊属性: `value`

3.注解的原理

三、元注解

1. `@Target`
2. `@Retention`

四、注解的解析

1. 步骤

一、概述

注解(`Annotation`)是Java代码里的特殊标记, 比如 `@Override`、`@Test`, 它们的作用是让其他程序根据注解信息来决定如何执行该程序。

注解可以用于类、接口、构造器、成员变量、方法、参数等位置处。

二、自定义注解

1. 格式

```
1 public @interface 注解名 {  
2     public 属性类型 属性名() default 默认值;  
3 }
```

代码示例如下:

- `MyAnnotation`:

```
1 public @interface MyAnnotation {  
2     String a();  
3     boolean b() default true;  
4     String[] c();  
5 }
```

- `Test1`:

```

1 package annotation;
2
3 @MyAnnotation(a = "zsh", c = {"html", "css", "js"})
4 public class Test1 {
5
6     @MyAnnotation(a = "zjl", b = false, c = {"Java", "Python"})
7     public void test1() {
8
9     }
10
11     public static void main(String[] args) {
12
13     }
14 }

```

2.特殊属性：value

如果注解中只有一个 `value` 属性或者其他属性都有默认值，那么使用注解时，`value` 字段名可以省略。

代码示例如下：

- `MyAnnotation2`：

```

1 public @interface MyAnnotation2 {
2     String value(); // 特殊属性
3 }

```

- `Test1`：

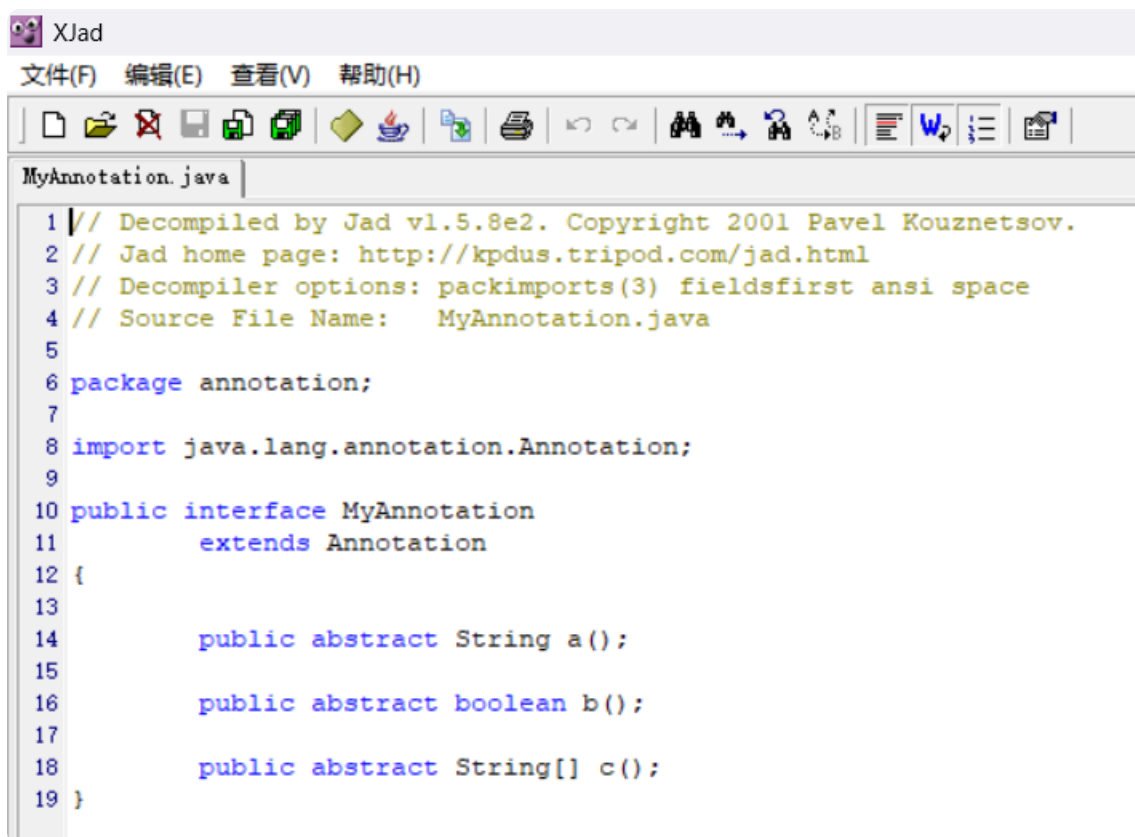
```

@MyAnnotation(a = "zsh", c = {"html", "css", "js"}) n
// @MyAnnotation2(value = "god")
@MyAnnotation2("god")
public class Test1 {

```

3. 注解的原理

我们执行一下 `Test1` 的 `main` 方法，然后使用 `XJad` 反编译工具对 `MyAnnotation.class` 字节码文件进行反编译并查看内容：



```
1 // Decompiled by Jad vl.5.8e2. Copyright 2001 Pavel Kouznetsov.
2 // Jad home page: http://kpdus.tripod.com/jad.html
3 // Decompiler options: packimports(3) fieldsfirst ansi space
4 // Source File Name:  MyAnnotation.java
5
6 package annotation;
7
8 import java.lang.annotation.Annotation;
9
10 public interface MyAnnotation
11     extends Annotation
12 {
13
14     public abstract String a();
15
16     public abstract boolean b();
17
18     public abstract String[] c();
19 }
```

分析反编译后的字节码文件，可知注解本质上是一个继承了 `Annotation` 这个注解接口的接口。

而我们在主程序中使用注解，其实相当于创建了注解的一个实现类对象。

三、元注解

Important

元注解就是修饰注解的注解。

1. @Target

作用： 声明被修饰的注解只能在哪些位置被使用。

格式： `@Target(ElementType.???, ElementType.???, ...)`

???	说明
TYPE	类和接口
FIELD	成员变量
METHOD	成员方法
PARAMETER	方法参数
CONSTRUCTOR	构造器
LOCAL_VARIABLE	局部变量

2. @Retention

作用： 声明注解的保留周期。

格式： `@Retention(RetentionPolicy.???, RetentionPolicy.???, ...)`

???	说明
SOURCE	只作用在源码阶段，编译为字节码文件后消失。
CLASS （默认值）	编译为字节码文件后继续存在，运行阶段消失。
RUNTIME （开发常用）	一直保留到运行阶段。

四、注解的解析

 Important

判断类、接口、构造器、成员变量、方法、参数等位置处是否存在注解，若存在则把注解里的内容解析出来。

1. 步骤

指导思想：要解析谁上面的注解，就应该先拿到谁。

`Class`、`Constructor`、`Field`、`Method` 都实现了 `AnnotatedElement` 接口，它们都拥有解析注解的能力。

序号	<code>AnnotatedElement</code> 接口提供的解析注解的方法	说明
01	<code>Annotation[] getDeclaredAnnotations()</code>	获取当前对象上的注解。
02	<code>T getDeclaredAnnotation(Class<T> annotationClass)</code>	获取指定的注解对象。
03	<code>boolean isAnnotationPresent(Class<Annotation> annotationClass)</code>	判断当前对象上是否存在某个注解。